UNIT - 5

Q.1. Explain B+ Tree and discuss search, insertion, and deletion algorithms with neat diagrams.

Answer :

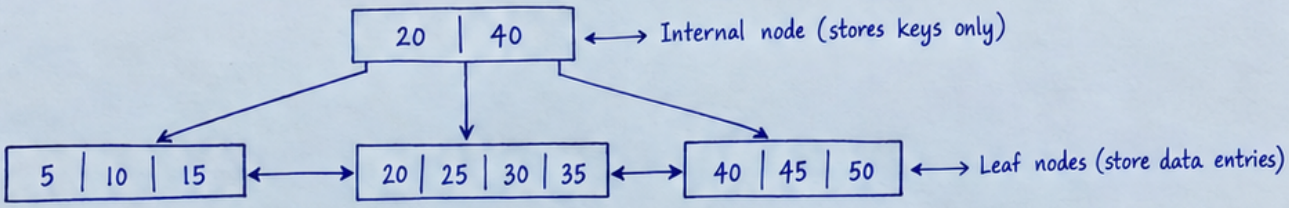
1. B+ Tree :

- B+ Tree is a balanced multi-level indexing structure used in DBMS.
- All actual data records (or record pointers) are stored only at leaf nodes.
- Internal nodes store only keys and child pointers.
- Leaf nodes are linked for efficient sequential access.

2. Properties of B+ Tree :

- It is a balanced tree.
- All leaf nodes are at the same level.
- Internal nodes contain search keys and pointers.
- Leaf nodes contain keys and record pointers.
- Leaf nodes are linked to each other.
- Supports efficient search, insertion, deletion, and range queries.

3. B+ Tree Diagram (Example of order 4) :



4. Search Algorithm :

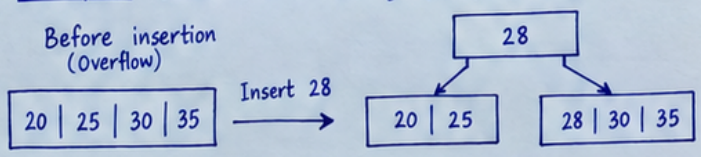
1. Start from the root.
2. Compare the search key with keys in the internal node.
3. Follow the appropriate child pointer.
4. Repeat until a leaf node is reached.
5. Search the key in the leaf node.
6. If found, return the record pointer; otherwise report not found.

Example : To search 25, from root [20 | 40] go to middle leaf and find 25.

5. Insertion Algorithm :

1. Search the correct leaf node for the new key.
2. Insert the key in sorted order in the leaf.
3. If the leaf has space, insertion ends.
4. If overflow occurs, split the leaf into two nodes.
5. Copy the first key of the new right leaf to the parent.
6. If the parent overflows, split the parent and continue upward.

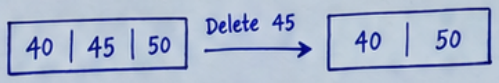
Example : Insert 28 in leaf [20 | 25 | 30 | 35].



6. Deletion Algorithm :

1. Search the leaf node containing the key.
2. Delete the key from the leaf.
3. If the node still has minimum required keys, stop.
4. If underflow occurs, first try redistribution from a sibling.
5. If redistribution is not possible, merge with a sibling.
6. Update the parent key.
7. If the parent underflows, continue upward.

Example : Delete 45 from leaf [40 | 45 | 50].



If a node becomes too small, borrow from sibling or merge.

7. Advantages of B+ Tree :

- Fast searching.
- Efficient insertion and deletion.
- Good for range queries.
- Used in file systems and database indexing.

8. Conclusion :

Thus, B+ Tree is an efficient dynamic indexing structure in DBMS. Its balanced nature and linked leaf nodes make search, insertion, deletion, and sequential access very efficient.

UNIT-5

Q.2. Explain Primary, Secondary, and Clustered Indexes with suitable examples.

Answer:

Indexes are data structures used in DBMS to improve the speed of data retrieval. They reduce the time required to search records in a table. Important types of indexes are Primary Index, Secondary Index, and Clustered Index.

1. Primary Index:

- A primary index is an index built on the primary key of a table.
- The data file is usually ordered according to the primary key.
- It is generally a sparse index because one index entry can represent a block of records.
- It is used when records are physically stored in sorted order of the primary key.

Example: STUDENT table is stored in ascending order of Roll_No.

STUDENT

Roll_No	Name
1	Ratan
2	Vivek
3	Lochan
4	Rithvik

Index

1	→	Block 1
3	→	Block 2

Here, Roll_No is the primary key, so the index on Roll_No is a primary index.

2. Secondary Index:

- A secondary index is an index created on a non-ordering attribute, that is, an attribute other than the primary key.
- The data file is not physically ordered according to this attribute.
- It is usually a dense index because each search-key value may need an index entry.
- It helps in searching records by non-key or alternate key attributes.

Example: In EMPLOYEE(Emp_ID, Name, Department), records may be stored by Emp_ID, but an index can be created on Department.

EMPLOYEE

Emp_ID	Name	Department
101	Arun	CSE
102	Banu	ECE
103	Charan	CSE
104	Divya	IT

Secondary Index on Department

CSE	→	101, 103
ECE	→	102
IT	→	104

Since Department is not the ordering field, this is a secondary index.

3. Clustered Index:

- A clustered index is an index used when records are physically stored in the order of a non-key field.
- Records having the same value of the clustering field are grouped together.
- Only one clustered index can exist on a table because data can be physically ordered in only one way.
- It is useful when many records share the same value of the indexing field.

Example: EMPLOYEE records are stored according to Department, so all CSE employees are together, all ECE employees are together, and so on.

Department	Emp_ID	Name
CSE	101	Arun
CSE	103	Charan
ECE	102	Banu
IT	104	Divya

Clustered Index on Department

CSE	→	First block of CSE records
ECE	→	First block of ECE records
IT	→	First block of IT records

Department is not unique, but records are grouped by it, so this is a clustered index.

4. Difference between Primary, Secondary, and Clustered Indexes:

Point	Primary Index	Secondary Index	Clustered Index
1. Based on	Primary key	Non-ordering attribute	Ordering non-key attribute
2. File order	Same as index field	Different from index field	Same as clustering field
3. Uniqueness	Usually unique	May be unique or non-unique	Usually non-unique
4. Index type	Often sparse	Usually dense	Can be sparse
5. Main use	Search by primary key	Search by alternate field	Group-based access

5. Advantages of Indexing:

- Speeds up searching.
- Reduces disk access time.
- Improves query performance.
- Helpful for large databases.

Conclusion:

Thus, primary, secondary, and clustered indexes are important indexing techniques in DBMS. They improve data retrieval efficiency in different ways depending on how the data is stored and searched.

UNIT-5

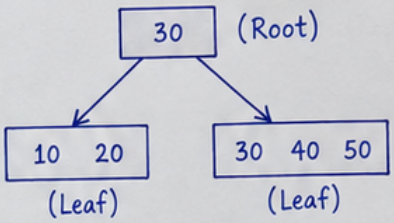
Q.3. Compare Tree-Based Indexing and Hash-Based Indexing.

Answer:

Indexing is used in DBMS to improve data retrieval speed. Two important indexing methods are Tree-Based Indexing and Hash-Based Indexing.

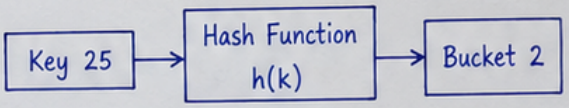
1. Tree-Based Indexing:

- Tree-based indexing uses a tree structure such as B-Tree or B+ Tree.
- Records are stored in sorted order.
- It supports equality search as well as range queries.
- Searching follows the path from root to leaf.
- It is suitable when ordered traversal is needed.
- **Example:** To find keys between 20 and 40, tree indexing can efficiently retrieve all records in that range.



2. Hash-Based Indexing:

- Hash-based indexing uses a hash function to map a search key to a bucket address.
- It is mainly used for exact-match searches.
- It does not maintain records in sorted order.
- Range queries are not efficient.
- Search is fast when the hash function distributes keys uniformly.
- **Example:** key 25 → h(25) → Bucket 2.



3. Comparison between Tree-Based and Hash-Based Indexing:

Point	Tree-Based Indexing	Hash-Based Indexing
Structure	Uses tree structure	Uses hash table / buckets
Order of keys	Stored in sorted order	Not stored in sorted order
Best for	Range queries and exact search	Exact-match search
Range queries	Efficient	Not efficient
Search method	Traversing tree nodes	Applying hash function
Sequential access	Possible	Not possible efficiently
Example structure	B-Tree, B+ Tree	Static hashing, dynamic hashing

4. Advantages:

Tree-Based Indexing:

- Good for range queries
- Supports ordered traversal
- Dynamic and balanced structure

Hash-Based Indexing:

- Very fast for equality search
- Simple direct access to bucket

Conclusion:

Thus, tree-based indexing is suitable for both exact and range queries, while hash-based indexing is best for exact-match searching. The choice depends on the type of queries used in the database.

UNIT-5

Q.4. Explain Hash-Based Indexing.

Answer:

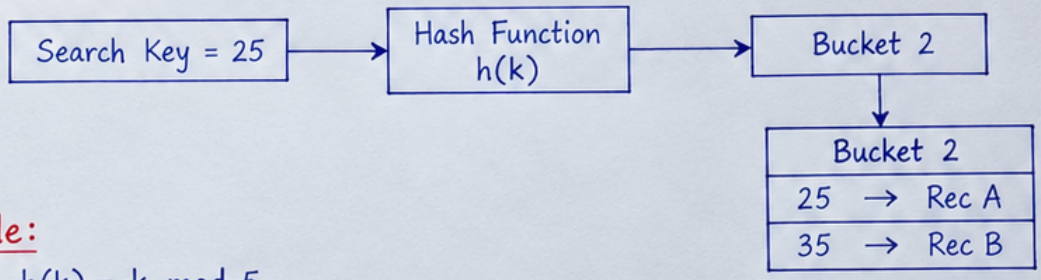
1. Hash-Based Indexing:

- Hash-based indexing is a technique used in DBMS to speed up record retrieval.
- It uses a hash function h(k) to map a search-key value k to a bucket address.
- The bucket stores the record or a pointer to the record.
- It is mainly useful for exact-match searches.

2. Working of Hash-Based Indexing:

1. Apply the hash function on the search key.
2. The function returns the bucket number or address.
3. Search the required record inside that bucket.
4. If many keys map to the same bucket, collision handling is required.

3. Basic Structure:



4. Example:

Suppose $h(k) = k \bmod 5$

Then:
- 20 → Bucket 0
- 12 → Bucket 2
- 27 → Bucket 2
- 34 → Bucket 4

Keys 12 and 27 go to the same bucket, causing a collision.

5. Collision Handling Methods:

- **Overflow bucket / chaining:** store extra records in linked overflow locations.
- Open addressing: search for another free location.

6. Types of Hashing:

- Static Hashing: Number of buckets is fixed.
- Dynamic Hashing: Number of buckets can grow or shrink as data changes.

7. Advantages:

- Very fast for equality search.
- Direct access to bucket.
- Simple and efficient for exact lookups.

8. Limitations:

- Not suitable for range queries.
- Performance depends on good hash function.
- Collisions may reduce efficiency.

9. Conclusion:

Hash-based indexing is an efficient indexing method for exact-match queries in DBMS. It uses a hash function to locate records quickly, but it is not suitable for ordered or range-based searching.

UNIT-5

Q.5. What is Query Optimization? Explain the various Query Optimization Algorithms in detail with suitable examples.

Answer:

1. Query Optimization:

Query optimization is the process of selecting the most efficient execution plan for a query from among many possible plans. Its aim is to reduce execution cost, response time, disk I/O, CPU usage, and memory usage.

2. Objectives of Query Optimization:

- Minimize query execution time.
- Reduce disk accesses and CPU cost.
- Choose efficient join order.
- Select suitable access path such as index scan or full table scan.
- Improve overall DBMS performance.

3. Basic Steps:

1. Parse the SQL query.
2. Convert it into relational algebra / query tree.
3. Generate alternative execution plans.
4. Estimate cost of each plan.
5. Choose the least-cost plan.

4. Various Query Optimization Algorithms / Techniques:

(a) Heuristic / Rule-Based Optimization:

Fixed rules are used to improve the query without calculating detailed cost.

Rules:

- Perform selection early.
- Perform projection early.
- Replace Cartesian product followed by selection with join.
- Execute the most restrictive selection first.
- Reorder joins to reduce intermediate results.

Example: For EMPLOYEE ⋈ DEPARTMENT with condition Dept = 'CSE', first select CSE rows from DEPARTMENT, then join with EMPLOYEE.

(b) Cost-Based Optimization:

The optimizer generates multiple plans and estimates their cost using statistics such as number of tuples, selectivity, indexes, and block accesses. The plan with minimum estimated cost is chosen.

Example: If only a few rows satisfy Roll_No = 10, index scan is cheaper than full table scan.

(c) Dynamic Programming Optimization:

This method is mainly used for join ordering. It finds the best plan for small subsets of relations and combines them to obtain the best overall plan.

Example: For relations R, S and T, compare (R join S) join T and R join (S join T), then choose the cheaper one.

(d) Greedy Optimization:

It chooses the locally best option at each step, such as the cheapest next join, without exploring all plans. It is faster but may not always give the global best plan.

Example: Among joins R join T, S join T and R join T, choose the pair with the lowest immediate cost first.

(e) Semantic Query Optimization:

Integrity constraints and semantic knowledge are used to simplify the query.

Example: If Age > 18 is always true for an EMPLOYEE table, that condition can be removed from the query.

5. Simple Example:

Consider the following SQL query:

```
SELECT Name
FROM   STUDENT
WHERE  Dept = 'CSE' AND Marks > 80;
```

First select STUDENT rows with Dept = 'CSE' and Marks > 80. If an index exists on Dept or Marks, use it to avoid scanning the whole table.

6. Advantages of Query Optimization:

- Faster query processing.
- Better resource utilization.
- Reduced response time.
- Efficient execution of complex queries.
- Important for large databases.

7. Conclusion:

Thus, query optimization is an essential part of DBMS that helps in choosing the best execution plan. Techniques such as heuristic optimization, cost-based optimization, dynamic programming, greedy approach, and semantic optimization improve the efficiency of query processing.

UNIT-5

Q.6. What is Database Backup and Recovery? Explain the various Backup and Recovery Techniques in detail with suitable examples.

Answer :

1. Database Backup and Recovery :

- Database Backup means creating a copy of database data so that it can be restored later if data is lost, corrupted, or damaged.
- Recovery means the process of bringing the database back to a correct and consistent state after a failure.
- Backup protects against data loss, while recovery helps restore normal database operation.

2. Need for Backup and Recovery :

- Protects data from hardware failure.
- Protects against software errors and accidental deletion.
- Helps recover from power failure, system crash, or disk failure.
- Ensures business continuity and data availability.

3. Types of Backup Techniques :

(a) Full Backup :

- A complete copy of the entire database is taken.
- It is simple to restore, but takes more time and storage.
- Example: On Sunday, copy the whole STUDENT database.

(b) Incremental Backup :

- Only the data changed after the last backup is copied.
- Saves storage space and backup time.
- Example: Full backup on Sunday, then Monday backup stores only Monday changes, Tuesday backup stores only Tuesday changes.

(c) Differential Backup :

- Copies all data changed since the last full backup.
- Restore is faster than incremental backup.
- Example: Full backup on Sunday; Wednesday differential backup contains all changes from Monday to Wednesday.

(d) Mirror / Duplicate Backup :

- An exact duplicate of the database is kept in another location or device.
- Useful for quick recovery and fault tolerance.
- Example: A company maintains a duplicate database on a secondary server.

4. Recovery Techniques in DBMS :

(a) Log-Based Recovery :

- The system keeps a log file of transactions.
- The log stores START, UPDATE, COMMIT, and ABORT records.
- During recovery, UNDO incomplete transactions and REDO committed transactions.
- Example:

  T_1 : START
  T_1 : Update A from 5000 to 4500
  T_1 : COMMIT

  If a crash occurs after commit, REDO T_1 if needed.

(b) Deferred Update :

- Actual database changes are postponed until the transaction commits.
- During recovery, only REDO committed transactions is needed.
- Example: If T_2 updates Balance but crashes before COMMIT, no database change is applied.

(c) Immediate Update :

- Database may be updated before commit.
- During recovery, both UNDO and REDO may be required.
- Example: If T_3 writes data and crashes before commit, those changes must be UNDONE.

(d) Checkpointing :

- At intervals, the DBMS saves a checkpoint in the log.
- Recovery starts from the last checkpoint, reducing recovery time.
- Example: If checkpoint is taken at 10:00 AM and crash occurs at 10:30 AM, recovery starts from the 10:00 AM checkpoint.

(e) Shadow Paging :

- The system keeps shadow pages as a backup copy of pages.
- Changes are made on new pages, while old pages are preserved.
- If failure occurs, the system can use the shadow copy.
- Example: If page P_1 is modified, the original page remains safe as shadow page.

5. Simple Example of Backup and Recovery :

Suppose a bank database stores account balances. A full backup is taken every Sunday. On Wednesday, the server crashes after some transactions. The DBMS restores the latest backup and uses the log file to REDO committed transactions and UNDO incomplete transactions. Thus the database becomes consistent again.

6. Difference between Backup and Recovery :

Point	Backup	Recovery
1. Meaning	Creating copy of data	Restoring data after failure
2. Purpose	Prevent data loss	Bring DB to consistent state
3. When used	Before failure	After failure
4. Output	Backup copy	Restored database

7. Advantages :

- Prevents permanent data loss.
- Improves reliability and safety.
- Helps in disaster management.
- Maintains database consistency.
- Essential for large organizations.

8. Conclusion :

Thus, backup and recovery are essential components of DBMS. Backup creates safe copies of data, and recovery restores the database after failure. Techniques such as full, incremental, and differential backup, along with log-based recovery, checkpointing, and shadow paging, help maintain data security and availability.